

Hardware Integration

1. RECEIVER SIGNAL PROCESSING :

Antenna → RF Front-End → Frequency Selection/Downconversion → ADC → Digital Signal Processing → VITA-49/VRT data interface => RECEIVER OBSERVATIONS

1. The **antenna** first captures the incoming analog RF signal
2. The **RF front-end** amplifies, filters, and tunes the required frequency range, after which the
3. Signal is **downconverted** to an appropriate intermediate/baseband frequency
4. Then passed to the **ADC** (Analog-to-Digital Converter), which converts it into digital samples
5. The **digital receiver**/FPGA/DSP then processes these samples and extracts useful information such as **frequency, bandwidth, power/SNR, timestamp, and detection/pulse information**
6. This processed signal data along with its metadata can then be formatted into **VITA-49/VRT** packets, which provide a standardized way to transport the digitized RF information and its context to the software layer.

Our smart scheduler consumes this information, decides which frequency to observe next.

VITA 49 (VRT) is a standard for communication between the **RF/receiver side and digital/software side** of a radio. It carries the actual received digital signal along with **metadata** describing it, such as RF centre frequency, bandwidth, sample rate, gain and timestamps. The idea is to create a common, hardware-independent way for RF receivers and digital processing systems to exchange information, which is particularly relevant to spectrum-sensing and signal-intelligence systems.

Once the receiver provides the *VITA-49/VRT data and metadata*, the flow moves into the *Smart Scheduler*

2. RECEIVER CONTROL :

RECEIVER OBSERVATIONS → Smart Scheduler → next tuning decision → Receiver API → Receiver retunes

1. The scheduler **receives** information such as the currently observed frequency, timestamp, signal/detection information, SNR, and whether the previous observation resulted in a hit or miss.
2. **Smart Scheduler** maintains the current state/history of the RF environment and uses its *RL/ML policy* to decide the next sensing action—for example, which frequency band should

be observed next and, in an extended design, how long to dwell there.

3. This **decision** is converted into a **tuning command**, conceptually such as **TUNE(frequency, bandwidth, dwell_time)**.

4. The command is then passed through a **receiver-control API/interface** provided by the SDR or receiver hardware.

5. The **receiver applies** the new tuning parameters, starts observing that portion of the spectrum

A **Receiver Control API** is the software interface that allows an application such as our **Smart Scheduler** to control the receiver's operating parameters. The scheduler sends commands such as **tune to a frequency, set bandwidth, change gain, or specify dwell time**, and the receiver applies those settings and begins the requested observation. There is **no single universal control API for all receivers**; it is usually provided by the receiver/vendor, although **VITA-49.2 also defines standardized control/command mechanisms** for supported systems.

3.OUR SOLUTION

RF Environment → Receiver → VITA-49/data interface → Smart Scheduler → tuning decision → Receiver API → Receiver → new observations

1.The **RF Environment** acts as the virtual real-world electromagnetic environment and contains the hidden ground truth about emitters, their frequencies, activity, timing, and behavior.

2. The **Virtual Receiver** interacts with this environment under realistic receiver constraints such as limited instantaneous bandwidth, tuning time, and dwell time.

3. The receiver's observations—such as detected pulses, frequency, timestamp, power/SNR, and hit/miss information—are passed through our **Receiver Interface**, which represents the role that a real **VITA-49/VRT-based data interface** could play in deployment.

4. The **Smart Scheduler** receives these observations and maintains the information available to it from previous scans. Its ML/RL policy then decides the next sensing action, such as *which frequency band to tune to and potentially how long to observe it*.

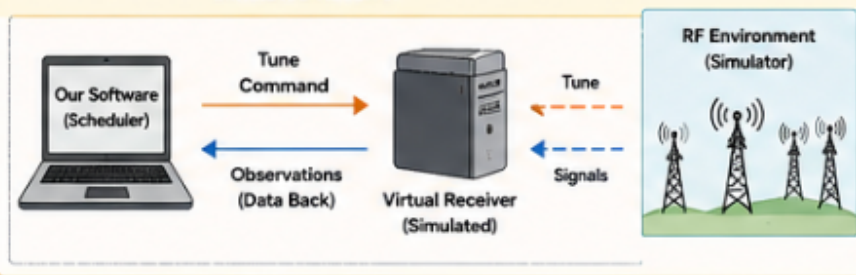
5. This decision is converted into a standardized internal command such as **TUNE(frequency, bandwidth, dwell_time)** and passed to a **Receiver Control API**.

(In the prototype, this API controls the Virtual Receiver; in a real system, the same scheduler layer could call the appropriate SDR/receiver vendor API to retune the hardware)

The important architectural point is that **our scheduler is kept independent of the RF environment and specific receiver hardware**, so the simulated receiver can eventually be replaced by a compatible real receiver without fundamentally changing the scheduling intelligence.

A. IN PROTOTYPE (What we use now)

Simulated Receiver



B. IN DEPLOYMENT (Future with real hardware)

Real EW Receiver / SDR



END-TO-END CLOSED LOOP

